AI Agent 团队架构

Harness Engineering in Action

内部技术培训 | 从跟 AI 聊天到管理工业级智能团队

01 | 目录

01 AI 工程演进：从 Prompt 到 Harness

02 Harness 架构：解决什么问题 & 核心优势

03 1+5 架构全景图（分层协作模型）

04 三大核心机制：验证、约束、进化

05 实施路线图 & 角色转变

为什么 Harness 是 10 倍效率的关键?

Prompt 工程

核心：Prompt 工程

痛点：结果不稳定，无法扩展

典型：帮我写个函数

Context 工程

核心：Context 工程

痛点：依然脆弱，难以维护

典型：参考这些文件写

Harness 工程

核心：Harness 工程

痛点：解法：约束、反馈、自动化

典型：Agent，按规则干活

比喻：模型是 CPU，Harness 是操作系统。

03 | 痛点 vs 解法

AI 不可控：经常脑补成功，交付结果不可靠

效率低下：人工反复确认，验证成本太高

知识孤岛：每次对话重新开始，经验不沉淀

扩展困难：增加功能需要重写所有 Prompt

确定性验证：代码级检查，不凭感觉

自动化流转：主调度 +5 专家，各司其职

仓库即大脑： Repo 是唯一的知识来源

持续进化：一人踩坑，全员免疫

04 | Harness 架构：从聊天到工业级控制

真实案例：老板说 帮我看看最近的销售数据

老板手动发 Excel 文件

老板手动复制结果到飞书

老板自己核对数据准确性

Main 自动识别 -> 路由给 Dataprocess

Dataprocess 读取 Excel + Schema 验证

运行 verify_schema.py 自动

数据流向

老板指令	飞书 / 语音输入
Main 调度	意图识别 -> 路由分发
5 专家并行	各司其职执行任务
验证层	Schema + verify_sch
结果交付	飞书 / JSON/PPT

05 | 架构全景图： 1+5 分层协作模型

Main 总调度中心

意图识别 -> 任务拆解 -> 路由分派 -> 验证签收 -> 最终交付

接收老板指令，智能分发任务给对应专家，最终验证签收后交付

日程	审查	日报	研究	数据
Assistant	Code Review	Daily Report	Research	Dataprocess
日程 / 邮件 / 杂活	架构 / 走读 / 审查	资讯 / 简报 / 日报	深挖 / 调研 / 方案	报表 / 数据 / 建模

验证层 | 硬约束 Schema 拦截 | 无验证不交付 | 错题本持续进化

06 | 角色职责矩阵

Agent	核心职责（Do's)	绝对禁忌（Don'ts)
Assistant	日程管理、邮件处理、日常杂活	深度技术、复杂数据
Code Review	架构建议、代码走读思路、审查策略	写生产代码、陷入细节
Daily Report	每日 AI/ 技术资讯抓取、简报生成	深度调研、处理私人数据
Research	垂直领域深挖（POCT/ 嵌入式 /Android)	处理 Excel 、写业务代码
Dataprocess	报表生成、数据库建模、数据清洗	写业务代码、无验证交付

07 | 核心机制一：验证优先

Iron Rule: 无代码验证，绝不交付

1. **执行任务** — Agent 执行具体操作 →
2. **运行脚本** — 运行 verify_file.sh / schema.py →
3. **获取证据** — 拿到 JSON: status=success →
4. **交付结果** — 带着证据链汇报

08 | 核心机制二：硬约束 > 软指令

Application Legibility: 让数据对 Agent 可读

Prompt: 请注意格式正确

结果： Agent 经常忽略或幻觉

问题： LLM 天生发散，不擅长守规矩

Schema: schemas/report_output.json

工具 : verify_schema.py 自动拦截

结果：不合规直接打回，无法蒙混过关

09 | 核心机制三：错题本与进化

每当 Agent 犯错，就工程化一个解决方案，让它再也不会犯同样的错。

-- Mitchell Hashimoto (Terraform 创始人)

任务失败或被老板指出

写入 LESSONS_LEARNED.md

全员启动时必读此文件

系统长记性，越用越强

10 | 标准工作流（SOP）

1
老板下达指令
意图 & 目标
2
Main 调度
路由 & 分派
3
专家执行
干活 + 自检
4
Main 签收
验证证据链
5
飞书交付
结果推送

11 | 场景：数据处理实战

核心价值：产出的数据不仅是给人看的，更是给其他 Agent 用的。机器可读，无缝流转。

12 | 场景：代码走读与审查

它不写生产代码，它只出主意。

关注点：逻辑闭环、架构风险、扩展性、团队规范。

典型任务： 10 万行遗留代码怎么 Review？

评审清单（Checklist）

架构改进建议

SonarQube 扫描策略

技术债务优先级排序

13 | 实施路线图

已落地

5 大专家 Agent 上线

验证优先机制落地

Harness 规则库建立

进行中

跨 Agent 自动化协作

研发到数据到报告自动流

定制化个人 Agent

目标

全自动驾驶模式

系统自我修复

研发效率 10 倍提升

从写代码的人变成管理代码 AI 的人

以前：我们一行行写代码，我们是瓶颈	现在：我们写规则、写约束、写验证脚本
以前：审查结果	现在：审查过程
以前：个人英雄主义	现在： Harness 工程师

让我们用 AI 重新定义研发效率

Q & A

架构已就绪，随时开干！

AI Agent 团队架构

Harness Engineering in Action

从跟 AI 聊天到管理工业级智能团队

01 | 目录

- AI 工程演进：从 Prompt 到 Harness
- Harness 架构：解决什么问题 & 核心优势
- 1+5 架构全景图（分层协作模型）
- 三大核心机制：验证、约束、进化
- 实施路线图 & 角色转变
- 实战避坑与最佳实践

02 | AI 工程演进

- 传统开发模式
  - Prompt 工程：写个函数
  - Context 工程：参考文件写
  - 痛点：结果不稳定，不可扩展
  - 瓶颈：人是唯一的控制器
- Harness 工程
  - 核心：约束 + 反馈 + 自动
  - 典型： Agent 按规则干活
  - 解法：工程化控制
  - 比喻：模型是 CPU ， Harness 是操作系统

03 | 痛点 vs 解法

痛点： AI 不可控

- 经常脑补，交付结果不可靠
- 效率低下：人工反复修改
- 无法规模化

解法：工业级 Harness

- 确定性验证：代码级检查
- 自动化流转：主调度 + 5 专家
- 结果：稳定、可预期、高效率

04 | 真实案例：老板的指令去哪了？

- 1. 输入：老板在飞书 / 语音说“帮我看看最近销售数据”
- 2. 意图识别： Main 自动识别需求，路由给 Dataprocess
- 3. 执行： DataAgent 跑脚本、查库、生成图表
- 4. 验证：运行 verify_schema.py ，检查数据格式
- 5. 交付：自动推送 JSON/PPT 到飞书，老板直接看结果

05 | 架构全景图： 1+5 分层协作

- Main (总调度中心): 意图识别 -> 任务拆解 -> 路由分派 -> 验证签收
- Assistant: 日程 / 邮件 / 杂活自动化
- Code Review: 架构走读 / 代码审查 / 风险拦截
- Daily Report: 资讯简报 / 自动日报 / 飞书推送
- Research: 深挖调研 / 方案设计 / 竞品分析
- Dataprocess: 报表生成 / 数据建模 / 格式转换
- 验证层 (铁律): 硬约束 Schema 拦截 | 无验证不交付

06 | 角色职责矩阵

- ● Main Orchestrator: 指挥官。不写代码，只负责调度与质量把控。
- ● Assistant: 贴身秘书。处理琐碎事务，释放开发者精力。
- ● Code Review: 质量门禁。专注架构合理性与潜在风险，不碰业务逻辑。
- ● Research: 军师。负责前沿技术调研，提供方案选型建议。
- ● Dataprocess: 数据工匠。处理 Excel/PDF/JSON 转换与建模。
- ● Daily Report: 播报员。每日定时汇总，保持信息同步。

07 | 核心机制一：验证优先

- Iron Rule: 无代码验证，绝不交付

- 1. 执行任务： Agent 生成代码 / 数据 / 文档
- 2. 运行脚本：调用 verify_schema.py 或 shell 脚本
- 3. 获取证据：拿到 JSON 结果 {status: 'success', data: ...}
- 4. 交付结果：带着证据链向 Main 汇报， Main 签收后交付用户

08 | 核心机制二：硬约束 > 软指令

- 软指令 (Prompt):
- • “请注意格式正确” -> 结果： Agent 经常忽略或脑补

- 硬约束 (Schema/Code):
- • `schemas/report_output.json` -> 强制校验字段类型、长度、必填项
- • 校验失败？直接打回重做，不凭感觉判断对错

- Application Legibility: 让数据对 AI 可读，而非对人可读

09 | 核心机制三：错题本与进化

- “每当 Agent 犯错，就工程化一个解决方案，让它再也不会犯同样的错。”

- 1. 任务失败 / 老板指出错误
- 2. 写入 LESSONS_LEARNED.md
- 3. 全员启动时必读此文件
- 4. 系统长记性，越用越强

- 进化不是靠训练，而是靠工程化约束与反馈循环。

10 | 标准工作流（SOP)

- 老板下达指令 -> 明确意图与目标
- Main 调度 -> 路由与分派给对应专家
- 专家执行 -> 干活 + 运行验证脚本自检
- Main 签收 -> 检查证据链，不合格直接打回
- 飞书交付 -> 结果推送到指定频道

- 循环：如果失败，触发“错题本”机制，更新约束后重试。

11 | 场景实战：数据处理

- 输入：老板发来杂乱的 Excel / PDF / 截图
- 处理 & 验证：
- ● DataAgent 提取 -> 清洗 -> 转换
- • 运行 Schema Check 确保字段完整
- 输出：标准 JSON / Excel / PPT

- 核心价值：产出的数据不仅是给人看的，更是给其他 Agent 调用的！

12 | 场景实战：代码走读与审查

- 它不写生产代码，它只出主意。
- 关注点：逻辑闭环、架构风险、扩展性、安全漏洞
- 评审清单 (Checklist):
- • 是否符合架构规范?
- • 是否有潜在的死锁 / 内存泄漏?
- • 日志与异常处理是否完善?
- ● SonarQube 扫描结果复核

13 | 实施路线图

- 已落地：
- ● 5 大专家 Agent 上线
- • 验证优先机制落地
- ● Harness 调度跑通

- 进行中：
- • 跨 Agent 自动化协作
- • 研发到数据到报告自动流

- 目标：

14 | 我们的角色转变

- 从“写代码的人”变成“管理代码 AI 的人”

- 以前：我们一行行写代码，我们是瓶颈
- 现在：我们写规则、写约束、写验证脚本

- 以前：审查结果
- 现在：审查过程

- 以前：个人英雄主义
- 现在： Harness 工程师

15 | 实战避坑与最佳实践

- 1. 保持 Prompt 极简：让 Agent 专注单一任务，不要塞太多上下文
- 2. 验证脚本先行：写代码前先写验证脚本，再让 AI 生成代码
- 3. 模块化输出： AI 的输出必须是结构化数据 (JSON/YAML)，方便下游调用
- 4. 小步快跑：每次只让 Agent 做一个小改动，验证通过再继续
- 5. 记录错题本：每一次失败都是系统进化的机会，务必记录

Q & A

架构已就绪，随时开干！
让我们用 AI 重新定义研发效率